Сборка: от чистой машины до работающего продукта

Закреплённые версии

Инструмент	Версия	Источник истины
Python	3.12	backend/Dockerfile, scripts/doctor.sh
Node.js	22.x	ci.yml
PostgreSQL	16-alpine	docker-compose.yml
Python-зависи-мости	точные пины в backend/requirements.lock.txt	make lock пересобирает lock из backend/requirements.txt
npm-зависимо-сти	lockfile	frontend/package-lock.json (npm ci)

Уровень A — исходный код

```
make doctor          # инструменты, .env, порты; ничего не меняет
make install         # зависимости backend (по lock-файлу) + frontend
make test            # pytest: backend + модуль аутентификации
make test-frontend   # tsc -b + vite build
make openapi-check   # спецификация совпадает с приложением (сравнение по смыслу)
make docs-check      # документация, ссылки, спецификации модулей
```

Секреты не нужны. POSTGRES_PASSWORD требуется только compose-уровням.

Уровень B — артефакты

```
make build-frontend  # frontend/dist (статика для nginx)
python -m compileall -q backend core.py main.py   # проверка синтаксиса пакета
```

Python-приложение не компилируется в бинарник: артефакт — образ (backend/Dockerfile) и статика frontend.

Уровень C — интеграционная среда

```
cp .env.example .env      # обязательно задать POSTGRES_PASSWORD
make compose-up
make smoke-test API_URL=http://localhost:8000
make create-user EMAIL=analyst@example.com
```

Уровень D — контейнерная сборка

```
docker compose build
docker compose up -d
./scripts/smoke-test.sh http://localhost:8080    # через web/nginx
```

Проверяется: non-root пользователь, HEALTHCHECK, fail-closed конфигурация (`POSTGRES_PASSWORD` без значения валит compose), отсутствие скачиваний в рантайме.

Уровень E — чистое развёртывание

```
make compose-down && docker compose down -v         # осознанно: удаляет данные
make compose-up && make smoke-test API_URL=http://localhost:8000
make backup
make restore FILE=<путь> && make smoke-test API_URL=http://localhost:8000
```

Автоматизировано в `.github/workflows/portability.yml`.

Переменные окружения

Полный список — `.env.example` (десятки параметров `AEZA_*` и `AUTH_*`). Секреты без дефолтов: `POSTGRES_PASSWORD`, `AUTH_PRIVATE_KEY`, `AEZA_SOURCE_INGEST_TOKEN`, `AEZA_METRICS_TOKEN`, `AEZA_TELEGRAM_BOT_TOKEN`, `AEZA_TELEGRAM_WEBHOOK_SECRET`.

Типовые ошибки

Ошибка	Причина	Решение
`mapping key "POSTGRES_PASSWORD" not found`	нет `.env`	`cp .env.example .env` и задать пароль
`UnavailableAnalytics` (503)	домен не инициализирован / пустой снимок	проверить `AEZA_SNAPSHOT_PATH` и `AEZA_SEED_DEMO_DATA`
`npm ci` падает	lockfile рассинхронизирован	править `package.json` и lockfile вместе
`make openapi-check` падает	маршрут изменён без регенерации	`make openapi`
UI не видит API	`AEZA_CORS_ORIGINS` не содержит адрес фронтенда	указать точный origin
AI не отвечает	`AEZA_AI_ENABLED=0` или провайдер недоступен	включить осознанно, проверить `/api/v1/model/*`

Доказательство сборки

Не «у меня работает», а зелёные `ci.yml` (уровни A–B) и `portability.yml` (уровни C–E). Матрица утверждений — docs/verification/matrix.md.

Что делает сборку воспроизводимой

Механизм	Где	Что гарантирует
Lock-файл Python-зависимостей	`backend/requirements.lock.txt`, `make lock`	одинаковые версии у всех и в CI
Lock-файл npm	`frontend/package-lock.json`, `npm ci`	одинаковое дерево фронтенда
Фиксированные образы	`postgres:16-alpine`, `node:22-alpine`, `python:3.12-slim`	нет «тихого» обновления базы
Детерминированная спецификация	`export_openapi.py` (`sort_keys=True`) + сравнение разобранного JSON	`openapi.json` не зависит от версии pydantic; повторная генерация ничего не меняет
Два снимка данных	`dataset/data.json` (официальный) и `dataset/data.demo.json` (демо)	приёмка не зависит от демо-данных
Явные обязательные секреты	`POSTGRES_PASSWORD` без значения по умолчанию в `.env.example`	стек не поднимется с демо-паролем
Ожидание готовности по HTTP	`scripts/wait-for-http.sh`	шаги CI не гоняются с приложением наперегонки